

AutoGen ve OpenClaw: bütün yüzey

İki agent sisteminin baştan sona anlatımı —
AutoGen'in her katmanını ve deseni, rakipleriyle yan
yana, ve OpenClaw'un harness'ının tamamı.

I · AutoGen

Üç katman · aktör modeli · iletişim ·
takımlar · beş bileşen · dokuz
desen

II · Rakipler

LangGraph · CrewAI · Agents SDK ·
Google ADK · Agno · DeepAgents

III · OpenClaw

gateway · protokol v4 · agent loop ·
oturum · hook · bellek · node ·
sandbox · prompt · SOUL · kuyruk ·
delegate

Bu belge nasıl okunur

Üç bölüm, üç ayrı soru. **I** AutoGen'in ne olduğunu ve her parçasının ne işe yaradığını anlatıyor. **II** aynı işi yapan diğer framework'lerle karşılaştırıyor. **III** OpenClaw'un harness'ını — yani bir agent'ı gerçekten çalıştırmak için gereken her şeyi — aynı ayrıntıda ele alıyor.

Her iddia etiketli, çünkü hepsi aynı ağırlıkta değil:

ÖLÇÜLDÜ	Bu makinede koşturuldu ya da kurulu pakete introspeksiyon yapıldı. Çıktı belgede.
KAYNAK	Birincil kaynakta yazıyor — resmî kılavuz, <code>.proto</code> dosyası, docstring, ya da reponun kendi kodu.
TEYİTSİZ	Okundu, doğrulanmadı. Çoğunlukla bu ortamda kurulu olmayan bir paket hakkında.

Bu ayrım süs değil. Bir agent framework'ünün belgeleri neyi *vaat ettiğini* yazıyor; bu belge nerede vaadin tutmadığını da yazıyor, ve ikisini karıştırmamak için etiket gerekiyor.

I · 1 – AutoGen nedir

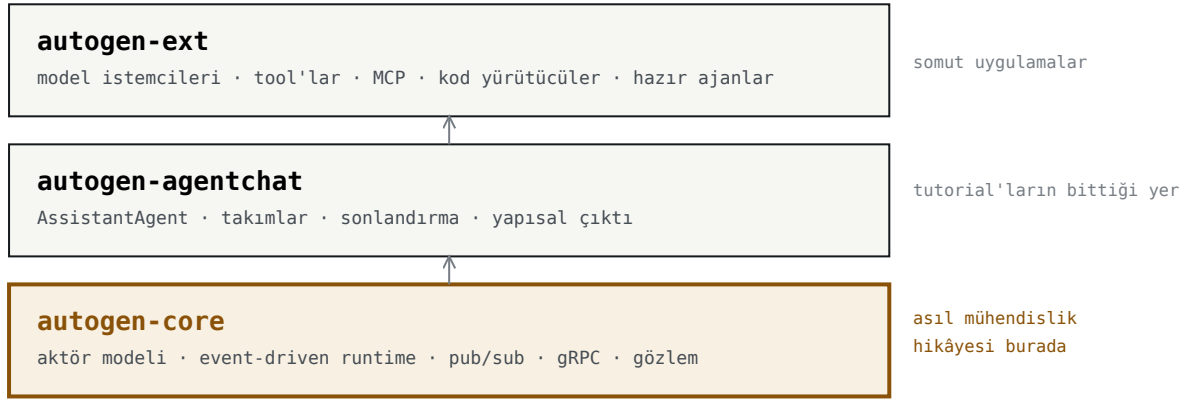
Microsoft'un çok-ajanlı uygulama framework'ü. Ama önemli olan şu: **bugünkü AutoGen (v0.4+), 2023'teki AutoGen'in devamı değil — sıfırdan yeniden yazımı.**

Gerekçeyi resmî göç kılavuzunun kendisi söylüyor [KAYNAK](#) :

"...we built AutoGen v0.4, a **from-the-ground-up rewrite adopting an asynchronous, event-driven architecture** to address issues such as **observability, flexibility, interactive control, and scale.**"

Bu cümledeki dört kelime — gözlemlenebilirlik, esneklik, etkileşimli kontrol, ölçek — tam olarak bir prototipin *üretime* geçerken çarptığı dört duvar. Bütün tasarımı açıklıyor: neden olay güdümlü, neden aktör modeli, neden müdahale kapısı var.

Üç paket, üç katman



Üç paket. Yukarıdan aşağıya somuttan soyuta.

Karşılaştırmaların çoğunun atladığı nokta: AutoGen'i ayıran şey üst katman değil, alt katman. AgentChat'te gördüğün `AssistantAgent` her framework'te var. `autogen_core` karşılığı çoğunda **yok**.

Bir uyarı, baştan

AutoGen bakım modunda. Microsoft yeni işi Microsoft Agent Framework'e (MAF) taşıdı. Bunun ölçülmüş bir bedeli var **ÖLÇÜLDÜ**: `autogen-ext 0.7.5` bağımlılığı `mcp>=1.11.0` diye **üst sınırsız** istiyor. MCP SDK 2.0 çıkınca kurulum `ImportError` ile kırıldı. Aynı gün `google-adk` aynı bağımlılığı `mcp>=1.24,<2` diye sınırlamıştı. *Aktif proje sınır koyar, bakım modundaki koymaz — ve düzeltecek kimse yoktur.*

Bu, AutoGen'i öğrenmenin değersiz olduğu anlamına gelmiyor. Aşağıdaki dokuz tasarım deseni ve protokol seçimleri AutoGen'in *dışında* yaşıyor. Ama üretime alacaksan bilerek al.

I · 2 – autogen_core: aktör modeli

Bu bölüm AutoGen'in kalbi. Buradaki beş kavram anlaşılmadan üst katmanların neden öyle davrandığı anlaşılmıyor.

2.1 Ajan bir nesne değil, bir adres

Çoğu framework'te ajan bir Python nesnesidir: yaratırsın, değişkende tutarsın, metodunu çağırırsın. AutoGen core'da öyle değil. Ajanın kimliği bir **çift**:

```
AgentId(type, key)
|         |
|         |----- hangi örnek → "company-42", "alice", "session-7"
|         |----- hangi rol/sınıf → "analyst", "session", "channel.web"
```

Ve sen sınıfı değil bir **fabrikayı** kaydediyorsun. Örneği runtime **ilk mesaj geldiğinde** yaratıyor — buna *lazy creation* deniyor:

```
await MyAgent.register(runtime, "my_agent", lambda: MyAgent("demo"))
#                                     ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
#                                     tip adı      fabrika (henüz çağrılmadı)
```

Neden böyle: "şirket başına bir analist ajanı" gibi bir şey istiyorsan, elle sözlük tutup örnek yaratman gerekmiyor. Adres verirsin, runtime örneği bulur ya da yaratır. Binlerce ajan tanımlamak, bir ajan tanımlayıp binlerce anahtar kullanmak demek.

2.2 Runtime – mesajları yürüten şey

TİP	NE ZAMAN	SINIF
Standalone	Tek süreç, tek dil	<code>SingleThreadedAgentRuntime</code>
Distributed	Çok süreç, çok makine, Python↔.NET	<code>GrpcWorkerAgentRuntimeHost</code> + worker'lar

Kritik vaat: **ikisi aynı API'yi sunuyor**. Ajan kodunu değiştirmeden tek süreçten dağıtık kurulumla geçebiliyorsun. **TEYİTSİZ** Dağıtık runtime bu ortamda koşturulmadı (`grpcio` kurulu değil), ama protokol tasarımı bunu destekliyor — ŞIII'te göreceğiz ki taşınan **Payload** mesajı `data_type` + `data_content_type` + `data` taşıyor, yani dilden bağımsız.

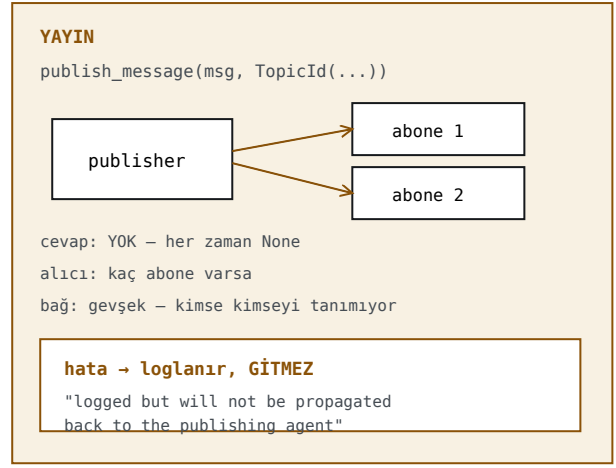
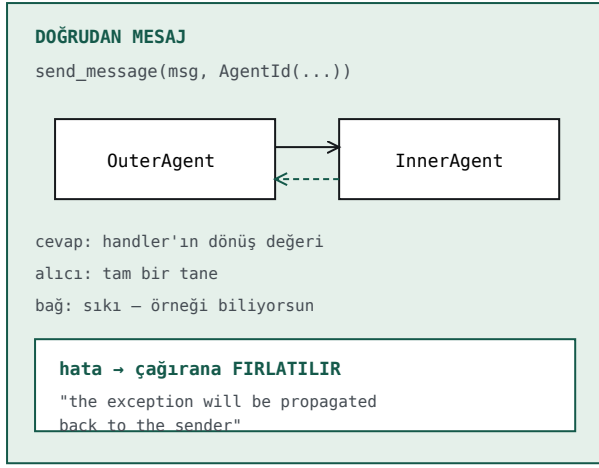
Durdurmanın üç yolu, üç farklı anlamı **KAYNAK**

<code>stop()</code>	Hemen döner — devam eden mesaj işlemeyi iptal etmez
<code>stop_when_idle()</code>	İşlenmemiş mesaj kalmayana kadar bloklar
<code>close()</code>	Kapatır, kaynakları bırakır

ÖLÇÜLDÜ `stop_when_idle()` bir **bariyer**, ve güvenilir değil. Çöken bir handler `_process_publish` içindeki `asyncio.gather` 'ı erken döndürüyor, hemen ardından `task_done()` çağırılıyor, kuyruk kardeşler hâlâ çalışırken "boşaldı" sayılıyor — ve **tamamlanmış sonuçlar sessizce kayboluyor**. Ne exception, ne uyarı. Üç koşuda birebir tekrarlandı.

2.3 İki iletişim biçimi – ve fark adresleme değil

Kılavuz net **KAYNAK**: "There are two types of communication in AutoGen core: **Direct Messaging**... and **Broadcast**." Üçüncü bir şey yok.



İki biçim yan yana. Asıl fark alt satırda: hatanın nereye gittiği.

Bu tek fark her şeyi açıklıyor. Paralel dallarda sonuç kaybediyorsan sebebi bir bug değil — yayın seçtiysen hata zaten kimseye gitmiyor. Doğrudan mesajda seçseydin her çöküş elinde olurdu, ama paralellik ve gevşek bağ giderdi.

Yayının dört sessiz kuralı [KAYNAK](#)

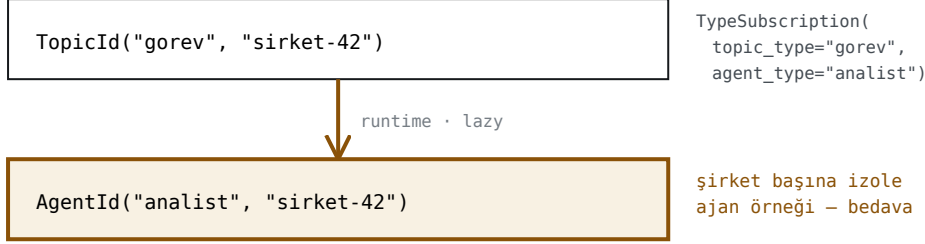
KURAL	PRATİKTE
"it will always return None " — ama yine de await edilmeli	O await bir senkronizasyon, cevap değil
"If a response is given... it will be thrown away "	Yayın handler'ından return yazan uyarı almaz
"...will not receive the message it published"	Kendi yayınını duymazsın — sonsuz döngüye karşı, açıkça
"...this will be logged but will not be propagated"	Çöküş kaybolmuyor; kontrol akışına girmiyor

Dördüncüsü en ince ayırım. "Yayında hata kaybolur" demek eksik: hata

`MessageHandlerExceptionEvent` olarak **olay akışına düşüyor**. Gözlemlenebilir, ama eyleme dönüştürülemez — yayınlayan taraf bir şey öğrenmiyor, ancak bir dinleyici öğrenebiliyor.

2.4 Topic ve abonelik – en atlanan sayfa

Yayının adresleme modeli. Bir topic'in iki parçası var: **tip** (kategori) ve **kaynak** (o kategorideki benzersiz kimlik). `TypeSubscription` bir topic *tipini* bir ajan *tipine* bağlıyor. Asıl incelik ise şu:



Topic kaynağı, ajan anahtarına dönüşür.

Çok kiracılı yapının hazır mekanizması — ve çoğu kişinin kaçırdığı yer

Kılavuzun saydığı üç senaryo:

SENARYO	NASIL
Tek kiracı, tek topic	Hepsi aynı topic tipi, kaynak sabit "default"
Tek kiracı, çok topic	Ajan başına ayrı topic tipi, kaynak aynı
Çok kiracı	Kaynak veriye bağlı — kiracı/oturum/kayıt başına izolasyon

`DefaultTopicId` kısayolu: topic tipi "default", **kaynak yayınlayan ajanın anahtarı**.

`@default_subscription` ile birlikte tek kapsamlı senaryoyu üç satıra indiriyor.

2.5 RoutedAgent – tipe göre yönlendirme

```
class MyAgent(RoutedAgent):  
    @message_handler  
    async def handle_text(self, message: TextMessage, ctx: MessageContext) -> None: ...  
  
    @message_handler  
    async def handle_image(self, message: ImageMessage, ctx: MessageContext) -> None: ...
```

Hangi handler'ın çalışacağını **tip anotasyonu** belirliyor — `if` bloğu değil, tip sistemi. Bir ajanın birden çok handler'ı olabilir.

Aynı tipi ayırmak: `match` **KAYNAK**

```
@message_handler(match=lambda msg, ctx: msg.source.startswith("user1"))  
async def on_user1(self, message: TextMessage, ctx: MessageContext) -> None: ...
```

Üç incelik, üçü de kolayca kaçıyor:

1. `match`, tip yönlendirmesine **ikincil** — önce tip, sonra koşul.
2. Koşullar **handler adlarının alfabetik sırasıyla** deneniyor. Sırayı dosyadaki yazım değil, fonksiyon adı belirliyor.
3. **Hiçbiri tutmazsa mesaj sessizce işlenmiyor**. Kılavuzun kendi örneğinde dört mesajdan biri hiç ele alınmıyor ve çıktıda buna dair tek satır yok.

ÖLÇÜLDÜ En sinsi tuzak burada. Tip çıkarımı `get_type_hints()` ile yapılıyor ve `from __future__ import annotations` varken anotasyonlar *modül genelinde* çözülüyor. `MessageContext` 'i fonksiyon içinde import edersen **kayıt sırasında** çıplak bir `NameError` alırsın — `handler`'ın kendisinde değil. Ayrıca parametre adları bağlayıcı: `message` ve `ctx` olmak zorunda.

2.6 Gözlemlenebilirlik – iki logger, altı olay

```
logging.getLogger(EVENT_LOGGER_NAME) # "autogen_core.events" – yapılandırılmış
logging.getLogger	TRACE_LOGGER_NAME) # insan okunur iz
```

OLAY	NE ZAMAN
<code>LLMCallEvent</code>	<code>create()</code> çağrıldığında
<code>LLMStreamEndEvent</code>	<code>create_stream()</code> bittiğinde — ve ilki asla
<code>ToolCallEvent</code>	Her <code>BaseTool</code> alt sınıfı çağrıldığında
<code>MessageEvent</code> · <code>MessageDroppedEvent</code>	Mesaj yolu
<code>MessageHandlerExceptionEvent</code>	Handler çöktüğünde

ÖLÇÜLDÜ İki tuzak, ikisi de sessiz. (1) Akış kullanınca sadece `LLMCallEvent` dinlersen maliyet **0** raporlanır — kod doğrudur, yanlış olayı dinliyordur. (2) `LLMCallEvent` alanlarını öznelik yapar ama `ToolCallEvent` hepsini `.kwargs` sözlüğünde tutar; `event.tool_name` yazarsan `AttributeError` alırsın ve o hata `logging.Handler` içinde olduğu için `handleError` tarafından **yutulur**. Olay hiç kaydedilmez, hata da görünmez.

2.7 InterventionHandler – teslimden önceki kapı

```
class TerminationHandler(DefaultInterventionHandler):
    async def on_publish(self, message, *, message_context):
        if isinstance(message, Termination):
            self._value = message
            return message # ya da DropMessage

runtime = SingleThreadedAgentRuntime(intervention_handlers=[TerminationHandler()])
```

Asıl kullanımı `DropMessage` döndürerek bir tool çağrısını **engellemek**. Neden değerli olduğu tek cümlede:

Onay kapısı ajanın uyum göstermeyi seçmesine değil, runtime'a dayanır. Model ne kadar ikna edilirse edilsin mesaj hattı onun altında.

ÖLÇÜLDÜ Ama bir bedeli var. `InterventionHandler` takmanın tek yolu `runtime`'ı kendin kurmak — ve o anda hata semantiği değişiyor: gömülü `runtime` `run_stream`'den *exception fırlatıyor*, dıştan verilen *hiç dönmüyor*. Gelmeyecek bir sonlanma mesajı bekleniyor; `MaxMessageTermination` kurtaramıyor çünkü yeni mesaj da gelmiyor. Duvar saati sınırı burada tedbir değil, **doğruluk şartı**.

I · 3 – autogen_agentchat: günlük iş

Core mesajlaşma modelidir; AgentChat onun üstünde *konuşma* soyutlaması kurar. Çoğu uygulama buradan başlar ve çoğu tutorial burada biter.

3.1 AssistantAgent – bütün kılavuzun ana kahramanı

Parametreleri, kararların çoğunu içeriyor:

PARAMETRE	NE YAPAR	DİKKAT
<code>model_client</code>	Hangi model	Ajan başına ayrı olabilir — model kademelendirme buradan
<code>tools</code>	Fonksiyonlar	Şema imzadan ve docstring'den üretilir
<code>workbench</code>	Tool <i>kaynağı</i> (MCP dahil)	<code>tools</code> ile birlikte verilemez
<code>system_message</code>	Rolü	—
<code>description</code>	Takım içinde yönlendirme bunu okur	Boş bırakılırsa seçici kör kalır
<code>model_context</code>	Bellek	Yoksa ajan turlar arası durumsuz
<code>output_content_type</code>	Pydantic ile yapısal çıktı	Takımda <code>custom_message_types</code> beyanı gerekir
<code>reflect_on_tool_use</code>	Tool sonucunu yorumlasın mı	—
<code>max_tool_iterations</code>	Zincirleme tool çağrısı	Varsayılan 1
<code>model_client_stream</code>	Token token akış	Kullanım olayını değiştirir (§2.6)
<code>handoffs</code>	Devir hedefleri (Swarm)	Tool adı küçük harfe düşer

ÖLÇÜLDÜ İki tuzak burada oturuyor. (1) `tools=` ile `workbench=` birlikte verilemez: *"Tools cannot be used with a workbench."* Çözüm, yerel fonksiyonları `StaticWorkbench` 'e sarıp `workbench` 'e **liste** vermek. (2) `max_tool_iterations` varsayılanı **1**: ajan bir tool çağırır, sonucu görür ve *susar*. "Önce ara, sonra bulduğunu incele" davranışı varsayılan ayarla imkânsız — ve hata vermiyor.

3.2 İki mesaj ailesi

BaseChatMessage – iletişim

Ajanlar arasında gidip gelen.

`TextMessage` · `MultiModalMessage` ·
`HandoffMessage` · `StopMessage` ·
`ToolCallSummaryMessage` · `StructuredMessage[T]`

BaseAgentEvent – iç olaylar

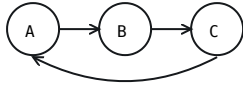
Ajanın yaptığı işin izi.

`ToolCallRequestEvent` · `ToolCallExecutionEvent`
· `ModelClientStreamingChunkEvent` ·
`MemoryQueryEvent` · `UserInputRequestedEvent`

Ayrım pratikte şuna yarıyor: akışı dinlerken *"bu bir cevap mı, yoksa ajanın yaptığı bir iş mi"* sorusunu **tip** cevaplıyor. Bir arayüzde tool çağrılarının ayrı satır olarak görünmesi bu sayede.

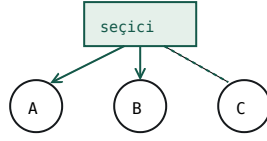
3.3 Beş takım – kim konuşacağına kim karar veriyor

RoundRobinGroupChat



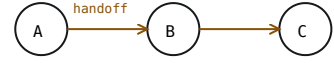
sıra · kimseyi atlayamaz · 274 token

SelectorGroupChat



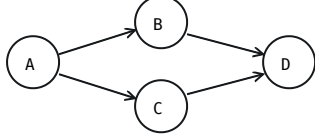
model ya da selector_func · 204 token

Swarm



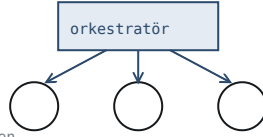
ajanın kendisi · devir bir tool · 334 token

GraphFlow



önceden çizili graf · fan-out + join · 270 token

MagenticOneGroupChat



genel amaçlı · plan + ilerleme defteri

%63,7 fark

Aynı görev, aynı ajanlar.
Değişen tek şey: yönlendirme.
Ödenen şey zekâ değil,
yönlendirme özerkliği.

ÖLÇÜLDÜ Token sayıları `poc/kiyas.py`'den — aynı görev, aynı ajanlar

SelectorGroupChat 'in iki kaçış kapısı önemli: **selector_func** ile seçimi Python'la yaparsın ve model hiç çağrılmaz — en ucuz yol; **candidate_func** ile modele sunulan aday listesini daraltırsın.

3.4 Sonlandırma – opsiyonel değil

MaxMessageTermination	N mesaj sonra
TextMentionTermination	Bir metin geçince ("TERMINATE")
TokenUsageTermination	Token sınırı
TimeoutTermination	Süre
HandoffTermination	Belirli bir hedefe devir olunca
SourceMatchTermination	Belirli bir ajan konuşunca
ExternalTermination	Dışarıdan <code>set()</code>
StopMessageTermination · FunctionCallTermination · FunctionalTermination	Mesaj tipi · tool adı · kendi fonksiyonun

MaxMessageTermination(10) | TextMentionTermination("TERMINATE") # birleştirilebilir

Sonlandırma koşulu olmayan takım = sonsuz döngü = gerçek fatura. Bu bir stil tercihi değil; bir ajan diğerine cevap verdiği sürece döngü kendi kendine bitmiyor.

3.5 Durum, bellek, insan

Durum: `save_state()` / `load_state()`. Kaydedilen şey `model_context` 'in tuttuğudur — bağlam vermediysen kaydedilecek sohbet de yok. İkisi birlikte düşünülmeli.

Bellek: Memory protokolü; uygulamaları `autogen_ext.memory` 'de — `chromadb`, `mem0`, `redis`, `canvas`.

İnsan döngüde: iki yol var, ve kılavuz ikincisini öneriyor. `UserProxyAgent` takımın içinde durup sırası gelince insana sorar — basit ama *takımı bloklar*. Diğer yol `HandoffTermination` ile çalışmayı bitirip geri dönmek: takım durur, sen cevabı alıp `run()` 'ı tekrar çağırırsın. Uzun bir insan beklemesi sırasında takımı ayakta tutmak kırılgan.

I · 4 – Components Guide: beş bileşen

Core kılavuzunun *Components Guide* başlığı altındaki beş sayfa. Bunlar "ajan nasıl düşünür"ün değil, "**ajan neyle çalışır**"ın cevabı: model, bağlam, araç, araç kaynağı, ve kod yürütücü.

4.1 Model Clients

Hepsi `ChatCompletionClient` protokolünü uyguluyor. Yerleşik olanlar:

İSTEMCİ	NE İÇİN
<code>OpenAIChatCompletionClient</code>	OpenAI modelleri ve OpenAI-uyumlu her endpoint (Gemini dahil)
<code>AzureOpenAIChatCompletionClient</code>	Azure OpenAI
<code>AzureAIChatCompletionClient</code>	GitHub Models ve Azure'da barınan modeller
<code>OllamaChatCompletionClient</code> (deneysel)	Yerel modeller
<code>AnthropicChatCompletionClient</code> (deneysel)	Anthropic
<code>SKChatCompletionAdapter</code>	Semantic Kernel AI connector'ları
<code>ReplayChatCompletionClient</code>	Test: yanıtlar önceden yazılı, ağ yok, deterministik
<code>ChatCompletionCache</code>	Herhangi bir istemciyi sarmalayıp yanıtları ön belleğe alır

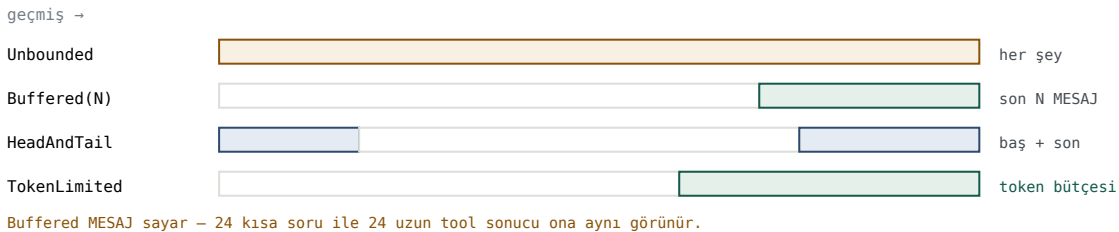
Sayfanın kapsadığı altı konu

Log Model Calls	<code>EVENT_LOGGER_NAME</code> 'e <code>LLMCall</code> olayı düşüyor — maliyet ölçümünün resmî yolu
Call Model Client	<code>create()</code> ile tek çağrı; <code>UserMessage</code> / <code>SystemMessage</code> listesi verirsın
Streaming Tokens	<code>create_stream()</code> — parça parça <code>str</code> , sonda <code>CreateResult</code>
Structured Output	<code>json_output=</code> parametresine Pydantic sınıfı verilir
Caching	<code>ChatCompletionCache</code> + bir <code>CacheStore</code> (disk ya da Redis)
Agent with a Model Client	Model istemcisi olan bir <code>RoutedAgent</code> 'ın en küçük hâli

ÖLÇÜLDÜ OpenAI-uyumlu her endpoint'te `model_info` zorunlu. Model adı OpenAI kataloğunda yoksa: `ValueError: model_info is required when model name is not a valid OpenAI model`. Ve bu bir *beyandır*, ölçüm değil — `function_calling`, `json_output`, `structured_output`, `vision`, `family` alanlarını sen doldurursun. Desteklenmeyen bir yeteneği iddia edersen hata başlangıçta değil, **yapısal çıktı isteyen ilk adımda** çıkar.

4.2 Model Context

Ajanın belleği: model istemcisine gönderilecek mesaj listesini yöneten nesne. Dört strateji, ve aralarındaki fark maliyeti doğrudan belirliyor:



Dört strateji. Maliyeti sınırlayan yalnız sonuncusu.

İki ince nokta. (1) `buffer_size` *mesaj* sayar, token değil — uzun bir sohbette maliyeti sınırlayan şey bu değildir. (2) `save_state()` 'in kaydettiği şey **bağlamın tuttuğudur**: bağlam vermediysen kaydedilecek sohbet de yoktur. Bu ikisi hep birlikte düşünülmeli.

4.3 Tools

Tool, modelin çalıştırabileceği bir kod parçası. AutoGen'de üç yol var:

Yerleşik tool'lar

`PythonCodeExecutionTool` gibi hazır olanlar — bir kod yürütücüyü tool hâline getiriyor.

Custom Function Tools

```
async def get_stock_price(ticker: str, date: Annotated[str, "YYYY/MM/DD"]) -> float:
    """Get the stock price."""
    return random.uniform(10, 200)

tool = FunctionTool(get_stock_price, description="Get the stock price.")
```

Şema imzadan ve docstring'den üretiliyor. Bunun pratik sonucu şu: *docstring dokümantasyon değil, arayüz*. Modelin tool'u doğru çağırıp çağırmayacağını docstring'in netliği belirliyor. `Annotated[str, "..."]` ile parametre başına açıklama da eklenebiliyor.

Calling Tools with Model Clients

Döngü elle de kurulabiliyor: modele tool şemalarını verirsin, model bir `FunctionCall` döndürür, sen çalıştırıp sonucu `FunctionExecutionResultMessage` olarak geri verirsin. Sayfa bunu adım adım gösteriyor — `AssistantAgent` 'ın kapağın altında yaptığı şey bu.

Tool-Equipped Agent

Ve üçüncüsü: `ToolAgent` 'a **doğrudan mesajla** tool çağrısı yollayıp cevabıyla bir *eylem-gözlem döngüsü* kurmak. Kılavuzun doğrudan mesajlaşma için verdiği kanonik örnek de bu (§2.3).

4.4 Workbench ve MCP

Tool bir *fonksiyon*; workbench bir **tool kaynağı**. Fark, tek metotta görünüyor:

```
await workbench.list_tools()          # "bana hangi tool'ların olduğunu söyle"
await workbench.call_tool(name, args)
```

Neden bu önemli: bir workbench, *ajan yazılırken var olmayan* tool'ları listeleyebiliyor. Uzak bir MCP sunucusunun yaptığı tam olarak bu — sunucu yarın yeni bir tool eklerse ajan onu yeniden yazılmadan görüyor. Sabit bir `tools=[...]` listesi bunu yapamaz.

SINIF	NE
<code>StaticWorkbench</code>	Yerel fonksiyonları workbench arayüzüne sarar
<code>McpWorkbench</code>	Bir MCP sunucusunun tamamı tek kaynak olarak
<code>StdioServerParams</code>	Yerel süreç (komut + argümanlar)
<code>SseServerParams</code>	Server-sent events
<code>StreamableHttpServerParams</code>	Streamable HTTP
<code>mcp_server_tools</code>	MCP tool'larını tek tek <code>FunctionTool</code> 'a çevirir

ÖLÇÜLDÜ tools= ile workbench= aynı ajana **verilemiyor**: *"Tools cannot be used with a workbench."* İkisini birleştirmenin yolu yerel fonksiyonları StaticWorkbench'e sarıp workbench'e **liste** vermek.

4.5 Command Line Code Executors

En basit kod yürütme biçimi: her kod bloğu bir dosyaya yazılır ve o dosya çalıştırılır. **Her blok yeni bir süreçte** koşuyor. İki uygulama, üç kullanım:

Docker

DockerCommandLineCodeExecutor
varsayılan imaj: python:3-slim
gereken: sh + python
önerilen yol

Local

LocalCommandLineCodeExecutor
doğrudan host'ta
kılavuzun kendi uyarısı:
izolesiz çalıştırmak riskli

Local + venv

sanal ortam içinde
bağımlılık izolasyonu var
süreç izolasyonu YOK

Docker context manager olarak kullanılabilir; değilse atexit konteyneri kapatıyor. auto_remove=False ile inceleme için bir

Üç seçenek, azalan izolasyon sırasıyla

Sayfa ayrıca **Docker içindeki bir uygulamanın Docker tabanlı yürütücü kullanması** senaryosunu da anlatıyor — yani konteyner içinden konteyner başlatma.

Bu sayfa AutoGen'in en ayırt edici yanı. Rakiplerin çoğunda kod çalıştırmak "bir tool"; AutoGen'de ayrı bir alt paket (code_executors), ayrı bir ajan (CodeExecutorAgent) ve kutudan çıkan bir onay kapısı (approval_func) var. Sebebi tarihsel: AutoGen'in kurucu makalesinin konusu kod yazma/çalıştırma döngüsüydü.

I · 5 – Dokuz çok-ajan deseni

Kılavuzun kendi ayrımı: akış **önceden mi çizili**, yoksa **konuşmadan mı doğuyor**. Ve bir uyarı — bunlar kütüphane değil, *desen*. `autogen_core`'la yeniden kurulabilir şeyler.

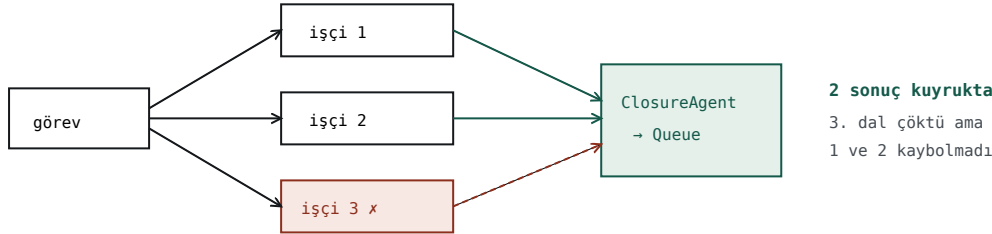
5.1 Concurrent Agents – ve toplama meselesi

Aynı topic'e abone ajanlar paralel koşar. Kritik parça **sonuç toplama**:

```
queue = asyncio.Queue[TaskResponse]()

async def collect(_agent: ClosureContext, message: TaskResponse, ctx: MessageContext) -> None:
    await queue.put(message)

await ClosureAgent.register_closure(
    runtime, "collector", collect,
    subscriptions=lambda: [TypeSubscription(topic_type=RESULTS, agent_type="collector")],
)
```



Güvenilmeyecek bariyer yok, çünkü bariyer yok.

Sonuç üretildiği anda yayınlanıyor; kuyruk onu çoktan tutuyor

MOTOR	TEMİZ	SARMALAYICI ARKASINDA HATA	HAM HATA
GraphFlow (AgentChat)	3 dal	2 dal	0-1 dal · süre sınırı dolar
pub/sub + ClosureAgent (core)	3 dal	2 dal	2 dal · ~3 ms

ÖLÇÜLDÜ aynı arıza enjeksiyonu · kaç dalın kaybolduğu **deterministik değil**

Ve bundan güçlü bir bulgu: resmî desenler bu konuda *birbiriyle çelişiyor*. *Concurrent Agents* kuyrukla topluyor; *Mixture of Agents* `asyncio.gather` ile — yani kaybın kaynağı olan yapıyla. Kılavuzun kendisi iki farklı arıza davranışı öneriyor.

5.2 Diğer sekiz desen

Sequential Workflow

Her ajan bir öncekinin çıktısını alır; `@type_subscription` ile her adım kendi topic tipini dinler. Kavram çıkar → yaz → biçimlendir.

AutoGen'in en zayıf temsil ettiği desen: ya `RoundRobinGroupChat` ya `GraphFlow` — ikisi de bu iş için ağır.

Group Chat

Ortak thread + bir konuşmacı seçici. Mesaj tipleri `GroupChatMessage` (içerik) ve `RequestToSpeak` (sıra sende).

Kılavuzun kendi cümlesi: *"not meant to be used in real applications... a starting point."* Üretim sürümü `SelectorGroupChat`.

Handoffs

OpenAI'nin Swarm desenine dayanıyor. Ajan bir **devir tool'u** çağırarak konuşmayı başkasına verir. Örnek: triyaj → onarım → insan.

ÖLÇÜLDÜ En pahalı desen: 334 token. Her devir bir tool çağırısı artı hiç iş üretmeyen bir LLM turu.

Mixture of Agents

Katmanlı işçiler: her katmanın çıktıları birleştirilip sonrakine verilir, sonunda bir orkestratör toplar.

Orkestratör `asyncio.gather` kullanıyor — §5.1'deki kaybın kaynağı olan yapı.

Multi-Agent Debate

Seyrek topoloji: her çözücü yalnız birkaç komşusuna bağlı (dört ajan bir kare, her biri iki komşuya). Birkaç tur boyunca komşuların cevaplarını görüp kendi cevabını düzeltir; sonunda bir toplayıcı **çoğunluk oyu** alır.

Maliyeti $N \text{ ajan} \times R \text{ tur}$. Ve oylama, aynı girdiye iki koşuda iki cevap verme riskini geri getirir.

Reflection

Üretici + eleştirmen döngüsü: `CoderAgent` yazar, `ReviewerAgent` bir `CodeReviewResult` döndürür, onaylanana kadar döner.

Kılavuz durma ölçütü önermiyor. Döngü kurarsan faturayı sınırlayan tek şey kalmıyor — bu, deseni olduğu gibi almanın en riskli yeri.

Code Execution

Group chat + kod yürütücü: model kod yazar, bir ajan çalıştırır, sonuç geri döner. AutoGen'in kurucu makalesinin konusu buydu.

Kod yürütme AutoGen'de **birinci sınıf**: ayrı alt paket (`code_executors`), ayrı ajan (`CodeExecutorAgent`), ve onay kapısı (`approval_func`). Rakiplerin çoğunda "bir tool".

Intro – desenlerin kendi sınıflandırması

Ana ayrım: **önceden çizili** (sequential, graph) ve **konuşmadan doğan** (group chat, handoff). Bu ayrım framework seçiminde de karşına çıkıyor — §II'ye bakın.

II · Rakiplerle karşılaştırma

Bir framework seçmek, aslında bir **metafor** seçmek. Her framework'ün bir dünya görüşü var ve o görüş bütün API'yi belirliyor.

TEYİTSİZ Dürüstlük notu: bu bölümdeki framework'lerden yalnız **Google ADK** bu ortamda kurulu. LangGraph, CrewAI, Agents SDK, Agno ve DeepAgents hakkındaki satırlar ya okumaya ya da o repoların kendi test/cookbook kodlarına dayanıyor — API yüzeyi için **KAYNAK** sayılır, davranış için sayılmaz.

II · 1 – Metafor eksen

FRAMEWORK	METAFOR	TEMEL PRİMİTİF
AutoGen	Konuşma	Ortak mesaj thread'i; ajanlar "conversable"
LangGraph	Graf	Düğüm + kenar + state; checkpoint
CrewAI	İnsan ekibi	Agent · Task · Crew (rol/backstory)
OpenAI Agents SDK	Devir	Agent · Tool · Handoff · Guardrail
Google ADK	Graf (derlenen)	Agent + workflow ajanları
Agno	İşletim sistemi	Agent · Team · Workflow + AgentOS
DeepAgents	Alt ajan delegasyonu	create_deep_agent + task tool'u

Pratik sonucu: AutoGen'de akış konuşmanın içinden doğuyor. Ajanlar birbirine mesaj *göndermiyor*, ortak bir thread'e yayın yapıyor ve herkes görüyor. "Kim konuşacak" kararı bir **strateji nesnesi** — değiştirilebilir.

Bu yüzden AutoGen'de tek kütüphane içinde beş desen var. Diğerlerinde çoğu zaman **desen = framework seçimi**: graf istiyorsan LangGraph, devir istiyorsan Agents SDK.

II · 2 – Asıl soru: bir ajan başka bir ajana nasıl iş verir

FRAMEWORK	BİRİNCİL BİRİM	AJAN → AJAN	HATA NEREYE GİDER
AutoGen	ajan	send_message / publish_message	doğrudan: çağırana · yayın: hiçbir yere
LangGraph	düğüm	Command(goto=...) + paylaşılan state	graf yürütücüsüne
Agents SDK	ajan	handoffs=[...] → tool çağırısı	çalıştıran döngüye
CrewAI	task	task çıktısı → sonraki task	crew döngüsüne
Google ADK	ajan ağacı	transfer_to_agent tool'u	graf motoruna
Agno	ajan / adım	takım lideri → üye	OnError ilkeli
DeepAgents	ajan	task tool'u (alt ajan)	tool sonucuna

Tablodan iki ders çıkıyor.

Birincisi: dördü (Agents SDK, ADK, DeepAgents ve AutoGen'in **Swarm**'ı) ajanlar arası geçişi *tool çağırısına* indirgemiş. Agents SDK'da handoff'lar tool'larla **aynı dönüştürücüden** geçiyor — modelin gözünde devir bir tool'dan ibaret. AutoGen'i ayıran şey, buna *ek olarak* bir mesaj otobüsü taşıması.

İkincisi ve daha keskini: son sütuna bakın.

AutoGen, "hata nereye gider" sorusuna iki farklı cevap veren tek framework. Diğerlerinde tek yol var — o yüzden sessiz kayıp da yok. Esneklik burada bedava değil.

II · 3 – DeepAgents'ın ters tercihi

Bir tanesi ayrıca dikkate değer. DeepAgents'ta alt ajan çağırmanın yolu `task` adında bir tool `KAYNAK`; yanında `start_async_task` / `check_async_task` çifti var.

Ne mesaj otobüsü var, ne graf, ne task listesi. Ana ajan bir tool çağırıyor, geriye **bir metin** alıyor. Amaç bağlam izolasyonu görünüyor: alt ajanın bütün ara adımları ana ajanın bağlamına girmiyor.

Bu, AutoGen'in ortak-thread modelinin **tam tersi** tercih. AutoGen'de herkes her şeyi görüyor — ve `poc/kiyas.py` 'de ölçtüğümüz bağlam şişmesi tam oradan geliyor.

II · 4 – Eksen eksen matris

EKSEN	AUTOGEN	LANGGRAPH	CREWAI	AGENTS SDK	GOOGLE ADK	AGNO
metafor	konuşma	graf	insan ekibi	devir	graf (derlenen)	işletim sistemi
iletişim	pub/sub ortak thread	state + <code>Command</code>	task zinciri	yalnız handoff	graf kenarları	lider → üye
akış kararı	strateji nesnesi	kenarlar	manager	ajanın kendisi	kenarlar (derlenen)	<code>Workflow</code> ilkelleri
runtime	aktör, dağıtılabilir	graf yürütücü	Python döngüsü	tur döngüsü	graf motoru	<code>Agent05</code> sunucusu
dağıtım protokolü	gRPC + CloudEvents	—	—	—	A2A	<code>Remote*</code> + AG-UI
tool protokolü	MCP (3 taşıyıcı)	MCP	MCP	MCP	MCP	MCP + geniş toolkit
izleme	OTel GenAI	LangSmith + OTel	ayrı	ayrı	<code>telemetry</code>	mlflow
bildirimsel tanım	yarım	—	YAML	—	tam	<code>Registry</code>
hata ne zaman görünür	çalışma zamanı	çalışma zamanı	çalışma zamanı	çalışma zamanı	graf kurulurken	çalışma zamanı
model arızası	—	—	—	—	—	<code>FallbackConfig</code>
durability	zayıf	checkpoint	katmanlı bellek	session	<code>Session</code>	<code>db=</code> vatandaş
insan onayı	kod için hazır	<code>interrupt()</code>	—	—	—	her ilkelde
kod yürütme	birinci sınıf	manuel	tool	tool	<code>code_executors</code>	bayrak
desen çeşitliliği	5 takım tipi	supervisor/swarm	seq/hier	tek	workflow ajanları	Step/Parallel/Loop/...
eval	ayrı araç	ayrı	ayrı	ayrı	içinde	ayrı
deploy	senin için	Platform	—	—	Vertex	<code>Agent05</code>
yaşam döngüsü	bakım modu	aktif	aktif	aktif	aktif	aktif

İki ince nokta

1 · Graf konusunda taraflar yer değiştirdi. AutoGen grafi `GraphFlow` olarak *beş takım tipinden biri* diye ekledi; ADK grafi **merkeze** aldı ve konuşma modelini hiç benimsemedi.

2 · "ADK oturmuş" demek yanlış olur. ADK 2.0 kırıcı bir sürümdü ve 1.x hattı paralel sürüyor. Bir yılda temel API'sini değiştirdi.

II · 5 – 2026'nın yakınsaması

Karşılaştırmaların çoğu farkları sayıp benzerlikleri atlıyor. Oysa bu alan ciddi biçimde yakınsadı ve artık şu sorular **ayırt edici değil**:

Tool çağırabiliyor mu?	Hepsi. Ve aynı biçimde: imza + docstring → JSON şema.
MCP var mı?	Hepsinde.
OpenTelemetry?	Hepsi o yöne gidiyor.
Yapısal çıktı?	Hepsinde Pydantic.
Bellek eklentisi, insan döngüde, uzak bileşen?	Hepsinde bir karşılığı var.

Kalan üç gerçek fark:

1. **Runtime modeli** — aktör mü, graf yürütücü mü, düz döngü mü
2. **Akış kararının nerede verildiği** — konuşmadan mı doğuyor, önceden mi çizili
3. **Operasyonel yüzey** — eval, deploy, session kutunun içinde mi

AutoGen birincide açık ara önde, üçüncüde açık ara geride. Bu tek cümle, "AutoGen'i ne zaman seçmeli" sorusunun çoğunu cevaplıyor.

II · 6 — Ne zaman hangisi

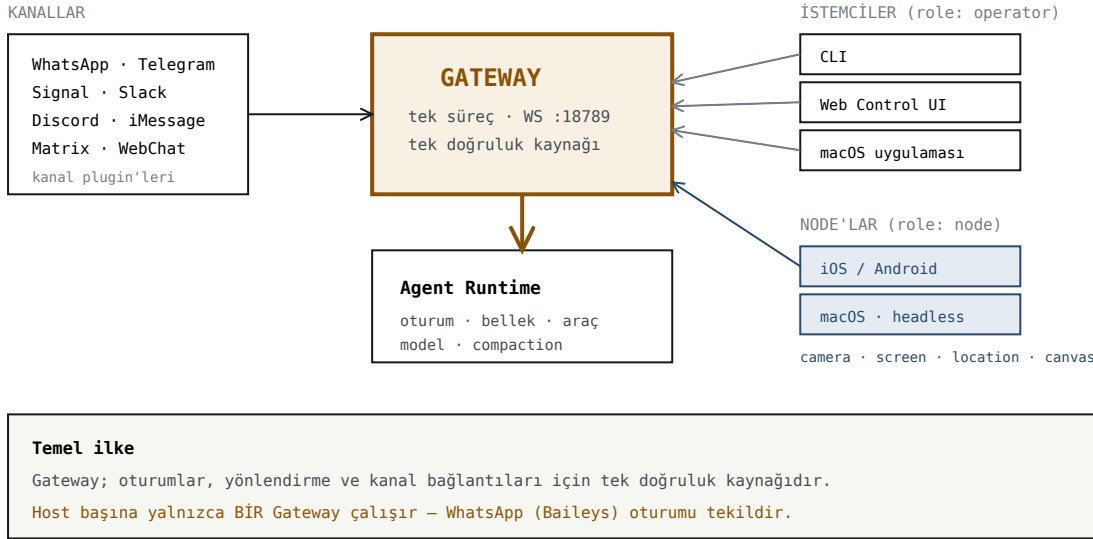
DURUM	TERCİH	GEREKÇE
Tek ajan + birkaç tool	Agents SDK	AutoGen'in runtime'ı boşa yük
Checkpoint / geri sarma şart	LangGraph	AutoGen'de durability zayıf
Hazır deploy + eval + session	ADK	Kutunun içinde geliyor
Sıralı/koşullu/döngülü akış + onay	Agno	Her ilkelde <code>HumanReview</code>
Sağlayıcı arızasına dayanıklılık	Agno	<code>FallbackConfig</code> hata tipine göre yedek
Öğrenme bütçesi düşük	CrewAI	En düşük giriş engeli
Bağlam izolasyonu / uzun görevler	DeepAgents	Alt ajanın ara adımları bağlama girmiyor
Başkasının ajanıyla konuşmak	ADK (A2A)	AutoGen bu problemi kapsamıyor
Çok makine, tek sistem	AutoGen	gRPC + CloudEvents katmanı
Desenleri kıyaslamak	AutoGen	Beşi aynı API'de; değişkeni izole edebiliyorsun
Kod yazma/çalıştırma döngüsü	AutoGen	Kod yürütme birinci sınıf

III · OpenClaw harness'ı

AutoGen bir framework; OpenClaw bir **harness** — yani bir agent'ı gerçekten çalıştırmak için gereken her şey. İki rakip değil, farklı katman: AutoGen "ajan nasıl düşünür" sorusunu, OpenClaw "ajan nasıl *yaşar*" sorusunu cevaplıyor.

MIT lisanslı, TypeScript, pnpm monorepo. **Kendi yerleşik agent runtime'ı var** — harici bir framework'e bağımlı değil.

III · 1 – Yüksek seviye: tek Gateway



Tek süreç, üç tür bağlantı: kanallar, operatörler, node'lar

III · 2 – Gateway protokolü v4

WebSocket, metin çerçeveleri, JSON yük. **İlk çerçeve mutlaka connect isteği olmalı** — aksi hâlde sert kapatma.

```
İstek : {type:"req", id, method, params}
Yanıt : {type:"res", id, ok, payload|error}
Olay : {type:"event", event, payload, seq?, stateVersion?}
```

El sıkışma – üç adım

1. Gateway ön-kimlik **challenge** yollar: {nonce, ts}
2. İstemci **connect** gönderir: protokol aralığı, **rol**, **kapsamlar**, caps, commands, auth, ve **imzalı cihaz kimliği**
3. Gateway **hello-ok** döner: features, snapshot, auth, policy

Ön-kimlik çerçeveleri **64 KiB** ile sınırlı; el sıkışma sonrası varsayılan 25 MB. Yan etkili metotlar (send , agent) güvenli tekrar için **idempotency key** ister.

Roller ve kapsamlar

operator Kontrol düzlemi istemcisi. Kapsamlar: operator.read · .write · .admin · .approvals · .pairing · .talk.secrets

node Yetenek sunucusu — kamera, ekran, canvas, system.run

Rezerve çekirdek metot örnekleri her zaman `operator.admin` 'e çözülür: `config.*`, `exec.approvals.*`, `wizard.*`, `update.*`.

Broadcast kapsam kapılama: sohbet/agent/tool-result çerçeveleri en az `operator.read` ister. **Bilinmeyen olay aileleri varsayılan olarak fail-closed** — yeni bir olay tipi, izin verilmiş sayılmıyor.

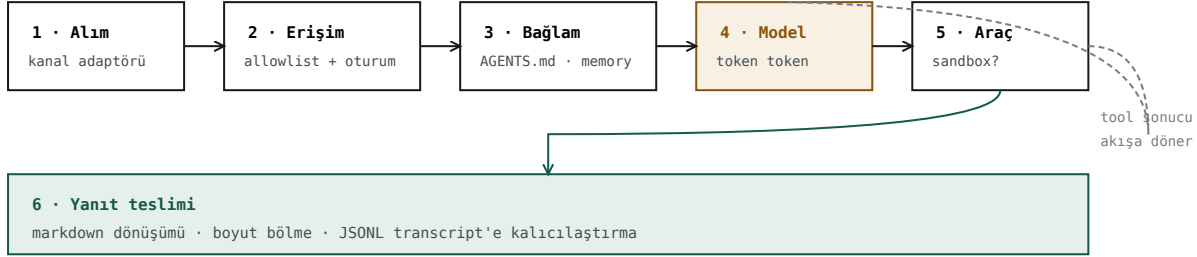
RPC metot aileleri

Sistem/kimlik	<code>health</code> · <code>status</code> · <code>system-presence</code> · <code>gateway.identity.get</code>
Model/kullanım	<code>models.list</code> · <code>usage.status</code> · <code>usage.cost</code> · <code>sessions.usage</code>
Kanallar	<code>channels.status</code> · <code>channels.logout</code> · <code>web.login.start/wait</code>
Oturum	<code>sessions.list/create/send/steer/abort/patch/reset/delete/compact</code> · <code>chat.*</code>
Agent	<code>agents.list/create/update/delete</code> · <code>agent.wait</code> · <code>audit.list</code> · <code>tasks.*</code>
Node	<code>node.pair.*</code> · <code>node.list/describe/invoke</code> · <code>node.pending.*</code>
Onaylar	<code>exec.approval.*</code> · <code>plugin.approval.*</code>
Otomasyon	<code>wake</code> · <code>cron.*</code> · <code>skills.*</code> · <code>tools.catalog/effective/invoke</code>
Sır/config	<code>secrets.*</code> · <code>config.get/set/patch/apply/schema</code> · <code>update.run/status</code>

Sürüm sabitleri `packages/gateway-protocol` 'de: `PROTOCOL_VERSION=4`, `MIN_NODE=3` — node'lar N-1 uyumluluk penceresi kullanabiliyor. Şemalar `TypeBox`'tan üretiliyor, oradan JSON Schema ve Swift modelleri.

III · 3 – Agent loop: bir mesajın yolculuğu

Oturum başına **seri** çalışan süreç: alım → bağlam kurulumu → model çıkarımı → araç yürütme → akış → kalıcılık.



Gecikme bütçesi (ikincil kaynak): erişim <10ms · oturum <50ms · prompt <100ms · ilk token 200–500ms · tarayıcı 1–3s

Altı faz. WhatsApp örneğinde ölçülen gecikme bütçesiyle

Kuyruklama – ve neden önemli

Çalıştırmalar **oturma anahtarı başına** (session lane) ve isteğe bağlı global lane üzerinden serileştiriliyor; bu araç/oturum yarışlarını önüyor. Transcript yazımları ayrıca **süreç-farkında, dosya-tabanlı bir yazma kilidiyle** korunuyor (varsayılan bekleme 60 sn).

RPC'nin şekli

agent RPC parametreleri doğrular, oturumu çözer, meta veriyi kalıcı hale getirir ve **anında** {runId, acceptedAt} döner. agent.wait ise bir runId üzerinde lifecycle end/error bekler ve {status: ok|error|timeout} döner.

Bu ayrım kopyalanmaya değer. Akış, run'ın kendisi değil — *bir görünümü*. Bir MCP tool çağrısı ya da bir cron işi, dakikalarca açık bir stream tutamaz; ama bir runId alıp sonra sorabilir.

III · 4 – Oturumlar

KAYNAK	DAVRANIŞ
Direkt mesajlar (DM)	Varsayılan olarak paylaşımlı tek oturum
Grup sohbetleri	Grup başına izole
Odalar / kanallar	Oda başına izole
Cron işleri	Her çalıştırmada taze oturum
Webhook'lar	Hook başına izole

DM izolasyonu çok kullanıcıli kurulumlarda kritik — session.dmScope: main (hepsi tek) · per-peer · per-channel-peer (önerilen) · per-account-channel-peer.

Yaşam döngüsü – üç ayrı zaman damgası

sessionStartedAt	Günlük reset — yapılandırılan yerel saatte (varsayılan 04:00)
lastInteractionAt	Idle reset — idleMinutes hareketsizlik sonrası
updatedAt	Bookkeeping

Üçünü birleştirmek ikisini birden bozar: uzun bir konuşma hiç yaşanmaz, sessiz olan yanlış saatte resetlenir. Ayrıca *heartbeat/cron/exec sistem olayları oturumu canlı tutmaz* — yoksa bir zamanlayıcı, hiç konuşulmayan bir oturumu sonsuza kadar diri tutardı.

Durum yeri: `~/.openclaw/agents/<agentId>/sessions/sessions.json` (indeks) + `<sessionId>.jsonl` (transcript). Bakım: `pruneAfter: 30d`, `maxEntries: 500`.

III · 5 – Hook sistemi: iki katman

Dahili (Gateway) hook'ları olay güdümlü scriptler — `agent:bootstrap`, komut hook'ları (`/new`, `/reset`, `/stop`).

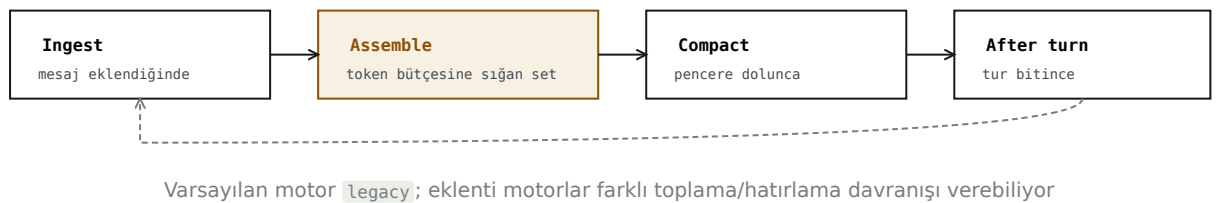
Plugin hook'ları ise boru hattının içinde:

HOOK	NE ZAMAN ÇALIŞIR
<code>before_model_resolve</code>	Oturum öncesi; provider/model'i deterministik override
<code>before_prompt_build</code>	Oturum yüklendikten sonra; <code>prependContext</code> enjekte
<code>before_agent_reply</code>	LLM çağrısından önce; turu üstlenip sentetik yanıt dönebilir
<code>agent_end</code>	Tamamlanma sonrası, nihai mesaj listesiyle
<code>before_compaction</code> / <code>after_compaction</code>	Compaction döngülerini gözler
<code>before_tool_call</code> / <code>after_tool_call</code>	Araç parametrelerini/sonuçlarını yakalar
<code>tool_result_persist</code>	Sonuçları transcript'e yazılmadan önce senkron dönüştürür
<code>message_received</code> / <code>message_sending</code> / <code>message_sent</code>	Gelen/giden mesaj
<code>session_start</code> / <code>session_end</code> · <code>gateway_start</code> / <code>gateway_stop</code>	Yaşam döngüsü sınırları

Karar kuralları, ve neden bu ikisi: `before_tool_call` → `{block: true}` **terminaldir**; `message_sending` → `{cancel: true}` **terminaldir**. Gerisi katkı verir. Yani sistemde tam iki nokta "hayır" diyebiliyor — biri araç kullanımına, biri dış dünyaya. Bu, uzatılabilirlikle güvenliği aynı mekanizmada tutmanın yolu.

III · 6 – Context Engine ve compaction

Her model çağrısında *hangi mesajlar, nasıl özetlenir, subagent sınırları arası nasıl yönetilir* sorularını cevaplayan katman. Dört yaşam döngüsü noktası:



Compaction'ın kuralları

- Eski turlar özetlenir, özet **transcript'e kaydedilir**, güncel mesajlar korunur.
- Araç çağrıları eşleşen `toolResult` girişleriyle birlikte tutulur** — bölme noktası bir araç bloğunun içine düşerse sınır kaydırılır.
- Auto-compaction varsayılan açık; limite yaklaşıncaya veya sağlayıcı overflow hatası dönünce (`compact + retry`) çalışır. OpenClaw düzinelerce sağlayıcıya özel overflow hata dizesini tanıyor.

- **Memory flush:** compaction öncesi sessiz bir tur çalıştırıp önemli notları diske yazabiliyor.

Compaction ≠ pruning. Compaction sohbeti özetler (kalıcı); pruning yalnız eski araç sonuçlarını kırpar (bellekte, istek başına).

`ownsCompaction: true` diyen bir motor kendi compaction'ını yönetir ve yerleşik olan devre dışı kalır. **Hata izolasyonu:** seçili plugin motoru yüklenemez ya da çökerse OpenClaw onu o Gateway süreci için *karantinaya alır* ve `legacy` 'ye düşer — **agent susmaz**.

III · 7 – Bellek

OpenClaw hatırlamayı **workspace'te düz Markdown dosyaları yazarak** yapıyor. Gizli durum yok.

DOSYA	NE ZAMAN OKUNUR	MALİYET
<code>MEMORY.md</code>	Oturum başında, prompt'a	Her turda ödenir → kısa kalmalı
<code>memory/YYYY-MM-DD.md</code>	Yalnız arandığında	Büyümesi bedava
<code>DREAMS.md</code>	Opsiyonel	"Dreaming" özeti

Araçlar: `memory_search` (hibrit: vektör benzerliği + anahtar kelime) ve `memory_get` (belirli dosya/satır aralığı). Backend'ler: builtin SQLite · QMD · Honcho · LanceDB.

Dreaming opsiyonel bir arka plan konsolidasyonu: kısa vadeli hatırlama sinyallerini toplar, puanlar, yalnız eşiği geçenleri `MEMORY.md` 'ye terfi ettirir.

Bu tasarımın değeri şu: okunabilen bellek düzeltilebilir, gözden geçirilebilir, silinebilir ve isterse sürüm kontrolüne girer. Okuyamadığın bir vektör deposu, yanlış bir olgunun sonsuza kadar yaşayabileceği yerdir.

III · 8 – Plugin ve capability sistemi

Genişletmenin kalbi **capability (yetenek) modeli**. Ve buradaki tek cümlelik ayırım, bütün sistemin uzatılabilirliğini açıklıyor.

plugin = sahiplik sınırı (bir şirketin ya da özelliğin tüm yüzeyi tek plugin'de) · **capability = çekirdek sözleşme** (birden çok plugin uygular ya da tüketir)

Yeni bir alan eklenirken ilk soru *"hangi sağlayıcı bunu hardcode etsin?"* değil, **"çekirdek yetenek sözleşmesi nedir?"** olmalı. Katmanlama:

Core capability	orkestrasyon · politika · fallback · tipli sözleşme
↑	
Vendor plugin	vendor API · auth · katalog
↑	
Channel/feature	yüzeyde sunum

Yetenek türleri

YETENEK	KAYIT METODU	ÖRNEK
Text inference	<code>api.registerProvider</code>	anthropic, openai
CLI inference backend	<code>api.registerCliBackend</code>	anthropic, openai
Embeddings	<code>api.registerEmbeddingProvider</code>	vektör plugin'leri
Speech / Realtime voice / Transcription	<code>registerSpeechProvider</code> vb.	elevenlabs, openai, google
Media understanding	<code>api.registerMediaUnderstandingProvider</code>	google, openai
Image / Music / Video generation	<code>registerImage/Music/VideoGenerationProvider</code>	fal, google, minimax
Web fetch / search	<code>registerWebFetchProvider</code> · <code>registerWebSearchProvider</code>	firecrawl, brave, google
Channel / messaging	<code>api.registerChannel</code>	matrix, msteams
Gateway discovery	<code>api.registerGatewayDiscoveryService</code>	bonjour

Plugin şekilleri: `plain-capability` (tek yetenek) · `hybrid-capability` (çok yetenek — ör. openai text+speech+media+image) · `hook-only` · `non-capability` (araç/komut/servis/route ama yetenek yok).

Yükleme boru hattı – dört katman

Manifest + keşif	openclaw.plugin.json + bundle manifestleri
↓	
Etkinleştirme + doğrulama	
↓	
Runtime yükleme	native plugin'ler IN-PROCESS yüklenir, merkezi registry'ye kaydolur
↓	
Yüzey tüketimi	araç · kanal · hook · route · CLI · servis

Güvenlik uyarısı, ve OpenClaw'un kendi belgesinden: native plugin'ler Gateway ile **aynı süreçte, sandbox'sız** çalışıyor. Yüklü bir native plugin çekirdek kodla aynı süreç-seviyesi güven sınırına sahip — kötü niyetli bir native plugin, OpenClaw süreci içinde **keyfi kod yürütmeye eşdeğer**. Bundle'lar (çoğunlukla skills) metadata/içerik paketi olarak daha güvenli.

III · 9 – Node'lar: cihaz yetenekleri

Node'lar aynı WS sunucusuna `role: node` ile bağlanıp bağlantı anında yetenek *iddialarını* bildiriyor:

caps	Yüksek seviye kategoriler — <code>camera</code> · <code>canvas</code> · <code>screen</code> · <code>location</code> · <code>voice</code> · <code>talk</code>
commands	Invoke allowlist'i — <code>camera.snap</code> · <code>canvas.navigate</code> · <code>screen.record</code> · <code>location.get</code>
permissions	Granüler anahtarlar — <code>camera.capture</code> · <code>screen.record</code>

Gateway bunları iddia olarak alıyor ve sunucu tarafı allowlist'lerle zorluyor. Eşleştirme cihaz-tabanlı; onay device pairing store'da yaşıyor.

Kamera örneği somut: `camera.list` · `camera.snap` (jpg, base64, `facing/maxWidth/quality/delayMs`) · `camera.clip` (mp4, 0.25–60 sn). **iOS/Android/macOS** node'larda — ve her platformda kullanıcı ayarıyla kapatılabiliyor (`CAMERA_DISABLED`). iOS'ta yalnız **ön planda**.

Offline node'lar için dayanıklı iş kuyruğu var: `node.pending.enqueue/drain`; bağlı node'lar için `node.pending.pull/ack`.

III · 10 – Güvenlik: katmanlı savunma

Sandbox – varsayılan kapalı

AYAR	DEĞERLER	VARSAYILAN
<code>sandbox.mode</code>	<code>off</code> · <code>non-main</code> · <code>all</code>	<code>off</code>
<code>sandbox.scope</code>	<code>agent</code> · <code>session</code> · <code>shared</code>	<code>agent</code>
<code>sandbox.backend</code>	<code>docker</code> · <code>ssh</code> · <code>openshell</code>	<code>docker</code>

Gateway süreci her zaman host'ta kalıyor; yalnız araç yürütmesi sandbox'a taşınıyor. Docker backend varsayılanı sıkı: `network: "none"` (egress yok), `readOnlyRoot: true`, `capDrop: ["ALL"]`. Bind mount'larda tehlikeli kaynaklar (`/etc`, `/proc`, `~/.ssh`, `~/.aws`, docker soketleri) varsayılan blokludur.

Dört katman, sırayla

1. **Tool policy** (allow/deny) — sandbox kurallarından *önce* uygulanıyor
2. **tools.elevated** — açık bir kaçış: `exec` 'i sandbox dışında çalıştırır. Global + per-agent kapı, *ikisi de* izin vermeli
3. **Exec approvals** — bir exec onay gerektirdiğinde Gateway `exec.approval.requested` yayınlar; operatör `resolve` ile çözer. **Prepare ile approve arasında komut/cwd değişirse çalıştırma reddedilir**
4. **Audit ledger** — `audit.list`: yalnız metadata. *Prompt, mesaj, araç argümanı ve çıktı saklanmaz.* 30 gün, 100.000 kayıt

Üçüncü maddedeki ayrıntı iyi bir fikir: onay *bir plana* veriliyor, bir eyleme değil. Onaydan sonra komutu değiştirmek onayı geçersiz kılıyor — yoksa "şunu çalıştırayım mı" ile çalıştırılan şey aynı olmak zorunda değildi.

Ağ güvenliği

Güvenli varsayılan: Gateway **loopback'te** kalır, uzak erişim SSH tüneli ya da Tailscale ile.

`gateway.tailscale.mode: serve` (tailnet-only HTTPS) · `funnel` (public HTTPS, **paylaşımlı parola**)

řart) · off . lan / tailnet 'e bind edilirse paylaşımı sır zorunlu.

III · 11 – Durum yerleşimi ve kontrol arayüzleri

YOL	İÇERİK
~/openclaw/openclaw.json	Config
state/openclaw.sqlite	Paylaşımlı runtime state
agents/<id>/agent/openclaw-agent.sqlite	Per-agent auth profilleri (API key + OAuth)
credentials/	Sağlayıcı/kanal kimlik bilgileri
agents/<id>/sessions/	Transcript'ler + sessions.json
workspace/	Varsayılan agent workspace (MEMORY.md burada)

Dört kontrol arayüzü, hepsi aynı tipli WS API'sine bağlanıyor: Web Control UI (Gateway kendisi sunuyor, ayrı sunucu yok) · CLI (Commander.js) · macOS menü çubuğu uygulaması · mobil node'lar.

III · 12 – Çok-agent ve MCP köprüsü

Tek Gateway sürecinde birden çok **izole agent**: her biri kendi workspace, state dizini ve oturum deposuna sahip. Gelen mesajlar **bindings** ile doğru agent'a yönlendiriliyor.

Yönlendirme deterministik, en spesifik kazanır: exact peer → parent peer → peer wildcard → guild+roles → guild → team → account → channel → default agent. Aynı kademede birden çok eşleşme varsa config sırasında ilk olan kazanır.

agentDir agent'lar arasında **asla** paylaşılmamalı (auth/oturum çakışması). Ve **agent-to-agent mesajlaşma varsayılan kapalı**, açıkça etkinleştirilip allowlist'lenmeli.

MCP – iki ayrı rol

KOMUT	OPENCLAW'UN ROLÜ	NE VERİR
openclaw mcp serve	MCP sunucusu	Kanal konuşmalarını dışa açar (stdio)
openclaw mcp add/set/...	MCP istemci kaydı	Üçüncü taraf sunucuları runtime'lara yansıtır

ÖLÇÜLDÜ mcp serve'ün gerçek yüzeyi dokuz tool: conversations_list · conversation_get · messages_read · messages_send · events_poll · events_wait · attachments_fetch · permissions_list_open · permissions_respond.

Sonucusu dikkat ister. permissions_respond, OpenClaw'un *kendi* bekleyen izin isteklerini cevaplıyor — yani insanın vermesi gereken onayları. Bir ajana verilirse, iki bağımsız kapı bire iner. Adında "send" gibi bir fiil yok; tahminle yazılmış bir engelleme listesi onu geçirir.

Ve stdio'nun iki sonucu: canlı olay kuyruğu bağlantıyla başlıyor ve *bağlantı kopunca yok oluyor* ("if the MCP client disconnects, there is no push target"); ayrıca her istemci kendi süreç kopyasını doğuruyor. Tek operatörlü bir araçta sorun değil, çok kullanıcı bir üründe ölçeklenmiyor.

III · 13 – Sistem promptu: üç katman, ve kim ne katabilir

OpenClaw her ajan koşusu için kendi sistem promptunu kuruyor — çalışma zamanında varsayılan prompt yok. Kurulum üç katmanlı:

<code>buildAgentSystemPrompt</code>	Saf renderer — açık girdilerden prompt üretir, global config'i doğrudan okumaz
<code>resolveAgentSystemPromptConfig</code>	Config'e bağlı düğmeler: sahip adı, TTS ipuçları, model alias'ları, bellek atıf modu, subagent delegasyonu
Runtime adaptörleri	Canlı gerçekleri toplar: tool'lar, sandbox durumu, kanal yetenekleri, bağlam dosyaları, sağlayıcı katkıları

Bu ayrımın amacı şu: *export/debug prompt yüzeyleri canlı koşularla hizalı kalıyor* — ve her runtime ayrıntısı tek bir dev builder'a dönüşmüyor.

Sağlayıcı katkıları – prompt cache sınırıyla birlikte

Bir sağlayıcı plugin'i, OpenClaw'ın sahip olduğu promptu *değiştirmeden* katkı yapabiliyor:

- Üç adlandırılmış çekirdek bölümden birini değiştirebilir: `interaction_style`, `tool_call_style`, `execution_bias`
- **Prompt cache sınırının üstüne** kararlı bir önek enjekte edebilir
- **Sınırın altına** dinamik bir sonek enjekte edebilir

Bu ayrım para demek. Prompt cache sınırının üstündeki kısım değişmediği sürece önbellekten okunuyor; altındaki her turda yeniden ödeniyor. Model ailesine özel ayarları "kararlı önek" olarak koymak, aynı içeriği dinamik sonekte taşımaya göre ölçülebilir biçimde ucuz.

Prompt'un sabit bölümleri: **Tooling** (yapılandırılmış tool'un tek doğruluk kaynağı olduğu hatırlatması) · **Execution Bias** (turda harekete geç, bitene ya da tıkanana kadar sürdür, zayıf tool sonucundan toparlan, değişken durumu canlı kontrol et, bitirmeden doğru) · **Safety** (güç arayışına ve gözetimi atlatmaya karşı kısa bir koruma).

III · 14 – SOUL.md: sesin yaşadığı yer

Belgenin kendi cümlesi net: *"if your agent sounds bland, hedgy, or corporate, this is usually the file to fix."*

SOUL.md'ye ne girer

Ajanla konuşmanın *hissini* değiştiren şeyler: ton, görüşler, kısalık, mizah, sınırlar, varsayılan doğrudanlık seviyesi.

Ne girmez

Hayat hikâyesi, changelog, güvenlik politikası dökümü, ya da davranışsal etkisi olmayan bir "vibe" duvarı. **Kısa uzun, keskin belirsizi yener.**

Gerekçe OpenAI'nin kendi prompt kılavuzuna dayandırılmış: yüksek seviye davranış, ton ve hedefler *yüksek öncelikli talimat katmanına* ait, kullanıcı turuna gömülmeye değil. OpenClaw için o katman `SOUL.md` — ve **sürümlenmesi** öneriliyor, bir kez yazılıp unutulması değil.

III · 15 – Workspace: ajanın evi

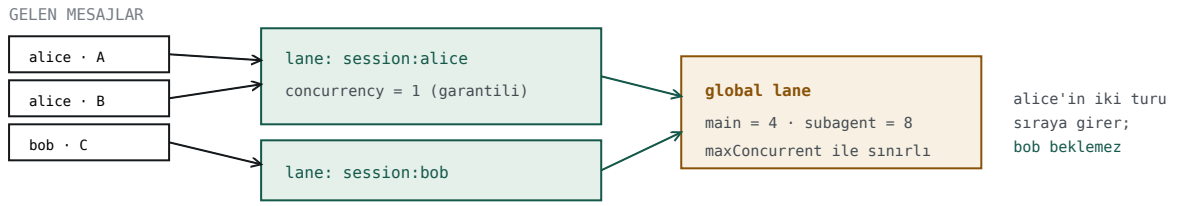
Workspace, dosya araçlarının kullandığı çalışma dizini ve bağlam kaynağı. `~/.openclaw/` 'dan **ayrı** — orası config, kimlik bilgileri ve oturumlar için.

Kritik uyarı, belgenin kendisinden: workspace varsayılan `cwd`'dir, **sert bir sandbox değil**. Araçlar görelî yolları workspace'e göre çözer, ama **mutlak yollar hâlâ host'un başka yerlerine ulaşabilir** — sandbox açık değilse. İzolasyon istiyorsan `agents.defaults.sandbox` gerekiyor.

Varsayılan `~/ .openclaw/workspace`; profil varsa `workspace-<profile>`; `OPENCLAW_WORKSPACE_DIR` ikisini de ezer. Varsayılan olmayan ajanlar `<state-dir>/workspace-<agentId>` 'ye düşüyor — **paylaşılan workspace'e değil**.

III · 16 – Kuyruk: lane'ler ve eşzamanlılık

OpenClaw gelen otomatik-yanıt koşullarını (tüm kanallar) küçük bir süreç-içi kuyruktan serileştiriyor. Amaç iki tane: pahalı LLM çağrılarının çakışmasını önlemek, ve paylaşılan kaynaklar (oturum dosyaları, loglar, CLI stdin) için yarışmayı bitirmek.



Yazma sırası beklerken bile **typing indicator hemen** tetikleniyor — kullanıcı deneyimi değişmiyor.
Varsayılanlar: mode "steer" · debounceMs 500 · ~2sn'den uzun bekleyen koşullar log'a not düşüyor.

Oturum başına garanti seri, oturumlar arası paralel — iki kademeli lane sistemi

III · 17 – Model failover: iki kademe

OpenClaw arızayı iki aşamada karşılıyor:

1. **Auth profil rotasyonu** — aynı sağlayıcı içinde, cooldown kurallarıyla
2. **Model fallback** — `agents.defaults.model.fallbacks` 'taki sıradakine

Ve bir ayrım var: yapılandırılmış varsayılanlar, cron işi primary'leri ve otomatik seçilmiş modeller fallback kullanabiliyor; ama **kullanıcının açıkça seçtiği oturum modeli *strict*** — sessizce başka modele düşmüyor. "Ben bu modeli istedim" ile "bir model çalışsın yeter" farklı niyetler, ve sistem ikisini ayırıyor.

III · 18 – Delegate ve commitments

Delegate – kurumsal mod

OpenClaw'ın varsayılanı *kişisel asistan*: bir insan, bir ajan. **Delegate** bunu kuruma taşıyor: ajanın **kendi kimliği** var (e-posta, görünen ad, takvim), insanların *adına* davranıyor ama **asla onları taklit etmiyor**.

Yetki, kimlik sağlayıcısının verdiği açık izinlerden geliyor; sınırlar `AGENTS.md` 'deki *standing orders*'da tanımlı — neyi kendi başına yapabilir, ne insan onayı ister.

Model, yönetici asistanının çalışma biçimi: kendi kimlik bilgileri, "on behalf of" gönderilen posta, ve tanımlı bir yetki kapsamı.

Commitments – kısa ömürlü takip hatıraları

Bir konuşmanın gelecekte bir kontrol fırsatı yarattığını fark edip hatırlamak. *"Yarın mülakatın var"* → sonrasında sorabilir. *"Çok yorgunum"* → sonra uyuyup uyumadığını sorabilir.

Ne MEMORY.md gibi kalıcı olgu, ne de tam bir hatırlatıcı. Bellek ile otomasyon arasında duruyor: konuşmaya bağlı bir yükümlülük hatırlanıyor, heartbeat zamanı gelince teslim ediyor.

Varsayılan kapalı (`commitments.enabled: false`).

Commitments, bu belgede anlatılan her şeyin arasında en az kopyalanabilir ama en çok düşündüren fikir. Çoğu agent sistemi "sorulanı cevapla" ekseninde; bu, ajanın *kendi açtığı döngüleri kapatmasını* bir mekanizma hâline getiriyor.

Kapanış – üç cümle ve bir liste

1 • AutoGen'i ayıran şey üst katman değil, alt katman

`AssistantAgent` her framework'te var. `autogen_core` karşılığı çoğunda yok — ve gerçek mühendislik problemlerini (izole oturum, olay akışı, müdahale kapısı, kuyrukla toplama) çözen orası.

2 • Aktör modeli runtime'ı koruyor, veriyi korumuyor

Ölçüldü, tekrarlandı, ve resmî desenlerin kendi arasında çeliştiği nokta. Sonuç: bariyere güvenme, **beklenen sonuç sayısını say**.

3 • Harness, framework'ten daha uzun ömürlü

OpenClaw'un çözdüğü şeyler — oturum yaşam döngüsü, hook noktaları, onay kapısı, compaction, denetim — hiçbir framework'ün problemi değil ve hiçbirini cevaplamıyor. AutoGen bakım moduna girse de bu sorular duruyor.

Bu belgede ölçülmemiş olanlar

- **LangGraph, CrewAI, Agents SDK, Agno, DeepAgents kurulu değil.** API yüzeyleri repo kodundan okundu; hiçbirini koşturulmadı.
- **Dağıtık AutoGen runtime'ı hiç çalıştırılmadı** — `grpcio` yok. Protokol bilgisi `.proto` descriptor'ından, davranış değil.
- **OpenClaw'un sandbox'ı, node'ları, delegate modu ve dreaming'i denenmedi.** Node listesi boş; kamera, prompt katmanları ve commitments belgeden okundu.
- **OpenClaw belgeleri 695 dosya; hepsi okunmadı.** Bu bölüm `concepts/` ve `gateway/` altındaki ana konulara dayanıyor — `platforms/`, `providers/`, `channels/` ve `specs/` ağırlıkla dışarıda.
- **Token ölçümleri tek koşumdan.** Model sıcaklığı ve endpoint yükü etkiliyor; `poc/kiyas.py` tekrar çalıştırılabilir.
- **Diğer framework'lerin fan-in davranışı ölçülmedi.** "LangGraph bu durumda daha iyi olurdu" diyemem — denemedim.

Kaynaklar — AutoGen: resmî core ve AgentChat kılavuzlarının tam metni (`docs/05`, `docs/08`), ölçülmüş tuzaklar (`docs/06`), protokol analizi (`docs/14`). OpenClaw: v2026.7.1-2 paket dokümantasyonu ve kaynak yapısı (`docs/13`), artı Paolo Perazzo'nun mimari anlatısı. Ölçümler: `poc/kiyas.py`, `pipeline/compare_fanin.py`.